PCG Arena: A Platform for Blind A/B Testing of Procedural Content Generators with Direct Preference Optimization

(PCG Arena: Platforma do Ślepego Testowania A/B
Generatorów Treści Proceduralnych
z Optymalizacją Preferencji)

Antoni Czapski

Praca magisterska

Promotor: dr Jakub Kowalski

Uniwersytet Wrocławski
Wydział Matematyki i Informatyki
Instytut Informatyki

March 2026

Abstract

Evaluating procedural content generators (PCGs) for video games poses significant methodological challenges, as player enjoyment is inherently subjective and difficult to quantify through automated metrics alone. This thesis presents PCG Arena, a web-based platform designed for blind A/B testing of Super Mario Bros level generators through human preference data. The platform enables players to play two procedurally generated levels side-by-side in their browser, vote for their preferred level, and optionally tag gameplay characteristics. Votes are aggregated using the Glicko-2 rating system, providing uncertainty-aware rankings with confidence intervals.

Building on the collected data, we conduct an extensive exploratory data analysis investigating what makes levels good, whether player preferences are consistent, and whether subjective tags correspond to measurable gameplay differences. These findings inform the development of a heuristic Judge Function achieving $r = 0.736$ correlation with human preferences. Finally, we present MarioDPO, a level generator fine-tuned with Direct Preference Optimization that achieves the highest win rate (83.3%) among all PCG methods tested, second only to hand-crafted original Nintendo levels.

Ewaluacja generatorów treści proceduralnych (PCG) dla gier wideo stanowi istotne wyzwanie metodologiczne, ponieważ przyjemność gracza jest z natury subiektywna i trudna do zmierzenia za pomocą automatycznych metryk. Niniejsza praca przedstawia PCG Arenę — platformę internetową do ślepego testowania A/B generatorów poziomów do gry Super Mario Bros poprzez dane o preferencjach graczy. Platforma umożliwia graczom rozgrywanie dwóch proceduralnie generowanych poziomów obok siebie w przeglądarce, głosowanie na preferowany poziom oraz opcjonalne tagowanie cech rozgrywki. Głosy są agregowane za pomocą systemu ratingowego Glicko-2, który zapewnia rankingi z przedziałami ufności.

Na podstawie zebranych danych przeprowadzono rozległą analizę eksploracyjną badającą, co czyni poziomy dobrymi, czy preferencje graczy są spójne oraz czy subiektywne tagi odpowiadają mierzalnym różnicom w rozgrywce. Wyniki te posłużyły do opracowania funkcji oceny (Judge Function), która koreluje z preferencjami ludzi na poziomie $r = 0.736$. Finalnie, praca przedstawia MarioDPO — generator poziomów wytrenowany metodą Direct Preference Optimization, który osiąga najwyższy wskaźnik wygranych (83.3%) spośród wszystkich metod PCG, ustępując jedynie oryginalnym poziomom Nintendo.

Contents

Chapter 1

Introduction

1.1 Procedural Content Generation in Video Games

tbd

1.2 The Evaluation Problem

tbd

1.3 Research Questions and Contributions

tbd

1.4 Thesis Structure

tbd

Chapter 2

Background and Related Work

2.1 Procedural Content Generation

tbd

2.1.1 Search-Based Approaches

tbd

2.1.2 Grammar-Based and Rule-Based Approaches

tbd

2.1.3 Machine Learning-Based Generation (PCGML)

tbd

2.2 Super Mario Bros as a PCG Benchmark

2.2.1 The Mario AI Framework

tbd

2.2.2 Mario Level Generators in the Literature

tbd

2.3 Evaluation Methods for Procedural Content

tbd

2.3.1 Automated Metrics

tbd

2.3.2 Agent-Based Testing

tbd

2.3.3 Human Evaluation Studies

tbd

2.3.4 Comparison of Approaches and Their Limitations

tbd

2.4 Rating Systems for Pairwise Comparisons

2.4.1 The Elo Rating System

tbd

2.4.2 The Glicko-2 Rating System

tbd

2.4.3 Arena-Style Evaluation Platforms

tbd

2.5 Preference Learning and Alignment

2.5.1 Reinforcement Learning from Human Feedback

tbd

2.5.2 Direct Preference Optimization

tbd

Chapter 3

PCG Arena — System Design

This chapter presents the design and implementation of PCG Arena, a web-based platform for blind A/B testing of procedural content generators for Super Mario Bros. We describe the system architecture, game engine, battle and voting mechanics, the novel AGIS matchmaking algorithm, the Glicko-2 rating integration, the builder submission workflow, and the deployment infrastructure.

3.1 Requirements and Design Goals

The primary goal of PCG Arena is to enable rigorous, human-centered evaluation of PCG algorithms through pairwise comparisons. Translating this goal into a concrete system imposes several requirements:

1. **Blind comparison.** Players must not know which generator produced which level, eliminating brand-loyalty bias and ensuring votes reflect genuine quality perception.

2. **Browser-based gameplay.** No installation should be required; the full Super Mario Bros experience must run in the browser with faithful physics.

3. **Uncertainty-aware ranking.** The rating system must quantify confidence in each generator's ranking, not merely produce a point estimate.

4. **Efficient matchmaking.** With a limited player population, every battle should maximise information gain toward stable rankings.

5. **Open submission.** Any researcher should be able to submit their own generator and receive a rating, with minimal friction.

6. **Rich telemetry.** Beyond the binary vote, the platform should capture gameplay trajectories, qualitative tags, and timing data for downstream analysis.

3.2 System Architecture

PCG Arena follows a three-tier web architecture with clear separation of concerns: a *presentation layer* (browser-based frontend), an *application layer* (RESTful backend), and a *data layer* (relational database). Table 3.1 summarises the technology choices for each layer.

Layer	Technology	Rationale
Frontend	React 18 + TypeScript 5.6 (Vite)	Canvas rendering; type safety
Backend	Python 3.12, FastAPI	Auto OpenAPI docs; Pydantic validation
Database	SQLite	Single-file; ACID; zero-config
Proxy	Caddy	Automatic TLS via Let's Encrypt
Container	Docker + docker-compose	Reproducible deployment

Table 3.1: PCG Arena technology stack.

3.2.1 Frontend

The frontend is a single-page application built with React 18 and TypeScript 5.6, bundled by Vite. Its source tree is organised into five top-level directories:

```
frontend/src/
  api/ # Backend communication (fetch wrappers)
  engine/ # Mario game engine (ported from Java)
  components/ # Reusable React UI components
  pages/ # Full-page route components
  contexts/ # React context providers (auth, theme)
```

Game rendering uses the HTML5 Canvas API for direct pixel manipulation, while the rest of the UI is standard React. Vite provides sub-second hot-module replacement during development and produces an optimised static bundle for production.

3.2.2 Backend

The backend is implemented in Python using FastAPI, chosen for its automatic OpenAPI documentation generation, native `async/await` support, and built-in request validation through Pydantic models. Key modules include:

- `main.py` — route definitions and middleware (CORS, rate limiting)
- `auth.py` — Google OAuth 2.0 and email/password authentication
- `glicko2.py` — Glicko-2 rating algorithm
- `matchmaking.py` — AGIS matchmaking algorithm
- `builders.py` — generator submission and management

- `stats.py` — analytics and data export endpoints

The API is versioned under the `/v1/` prefix and exposes endpoints for battles, votes, statistics, authentication, builder operations, and admin management. Administrative endpoints additionally require either session-based admin access or an API key.

3.2.3 Database

SQLite was selected as the database engine due to its single-file storage (simplifying backups and deployment), ACID transaction guarantees, and zero-configuration operation. The schema is managed through 17 sequential SQL migrations and comprises the following tables:

Table	Purpose
`generators`	PCG algorithm metadata and ownership
`levels`	ASCII tilemap content with dimensions and content hash
`battles`	Pairwise comparison instances (left/right level pairing)
`votes`	User preference outcomes with telemetry and tags
`ratings`	Current Glicko-2 state (μ, RD, σ) per generator
`rating_events`	Audit log of all rating changes
`generator_pair_stats`	Confusion matrix data (pairwise win/loss counts)
`users`	Authenticated user accounts
`user_sessions`	Active login sessions
`email_verifications`	Email verification tokens
`password_resets`	Password reset tokens
`player_profiles`	Anonymous player identities
`player_sessions`	Player session tracking
`level_stats`	Aggregate per-level gameplay statistics
`play_trajectories`	Detailed player position data for heatmaps
`level_features`	Structural feature extraction (tiles, enemies, gaps)
`schema_migrations`	Migration bookkeeping

Table 3.2: Database schema: all 17 tables.

3.3 Game Engine

A critical component of PCG Arena is the browser-based Super Mario Bros engine, which was ported from the Java-based Mario AI Framework [2] to TypeScript. The port required faithful preservation of gameplay physics while adapting rendering and input handling to browser constraints.

3.3.1 TypeScript Port

The porting process involved four main translation steps:

1. **Class structure.** Java classes were converted to TypeScript classes, preserving inheritance hierarchies (e.g. `MarioSprite` → `Mario`, `Enemy`).
2. **Physics constants.** All numerical constants were kept identical to the Java originals (Table 3.3).
3. **Rendering.** Java Swing/AWT rendering was replaced with the HTML5 Canvas API (`CanvasRenderingContext2D`).
4. **Input.** Keyboard event handling was reimplemented using browser `KeyboardEvent` listeners.

The key engine modules are:

```
engine/
  MarioGame.ts # Game loop (requestAnimationFrame)
  MarioWorld.ts # World state, collision, game logic
  MarioLevel.ts # ASCII tilemap parser
  MarioSprite.ts # Base sprite class (physics, rendering)
  sprites/ # Mario, enemies, items, projectiles
  graphics/ # Camera, sprite sheets
  effects/ # Particle effects
```

Parameter	Value
Ground inertia	0.89
Air inertia	0.89
Gravity	3.0
Jump velocity	−1.9
Walk speed	0.6
Run speed	1.2

Table 3.3: Core physics constants preserved from the Java Mario AI Framework.

The game loop runs at 30 ticks per second, matching the original Java framework. Rendering runs separately at 60 FPS via `requestAnimationFrame`, interpolating between physics frames. This decoupling preserves the exact physics behaviour of the original framework (which assumes 30-tick updates) while providing visually smooth animation in the browser. Physics updates are fully deterministic, ensuring consistent gameplay across sessions.

3.3.2 Level Format

Levels are stored as plain-text ASCII tilemaps, where each character encodes a tile type or entity spawn point. The format supports 37 distinct tile characters, summarised in Table 3.4.

Char	Meaning	Char	Meaning
`-`	Air (empty)	`Q`	Question block (coin)
`X`	Solid floor	`!`	Question block (coin, alt)
`S`	Brick block	`C`	Coin block
`#`	Pyramid block	`U`	Mushroom block
`D`	Used block	`L`	1-Up block
`%`	Jump-through platform	`1`	Invisible 1-Up block
`|`	Platform background	`2`	Invisible coin block
`?`	Question block (mushroom)	`o`	Free-standing coin
`@`	Question block (mushroom, alt)	`M`	Mario spawn
`E`	Goomba	`F`	Flag (level exit)
`g`	Goomba (alt)	`t`	Pipe (empty)
`G`	Winged Goomba	`T`	Pipe (flower)
`k`	Green Koopa	`<`	Pipe top left
`K`	Winged Green Koopa	`>`	Pipe top right
`r`	Red Koopa	`[`	Pipe body left
`R`	Winged Red Koopa	`]`	Pipe body right
`y`	Spiky	`*`	Bullet Bill body
`Y`	Winged Spiky	`B`	Bullet Bill head
		`b`	Bullet Bill neck

Table 3.4: Complete ASCII tile legend (37 characters).

A typical level file consists of 16 rows (the default height) and up to several hundred columns, stored with newline-separated rows. For example, the original Super Mario Bros World 1-1 is encoded as a 202×16 character grid. Figure 3.1 shows the opening segment (first 50 columns) of this level:

```
--------------------------------------------------------------------------------...------
--------------------------------------------------------------------------------...------
--------------------------------------------------------------------------------...------
-------------------------------oo-----------------------------------------------...------
---------------------------------------g-g--------------------------------------...------
----------------------ooor---------%%%%%%%-------------oooo----------R-----------...------
--------------------%%%%%----------|||||--------------%%%%----------------g----o...------
----------------------|||-----------|||||----oo---SSSS--||-------------%%%%%%----...------
----------------------|||-----------|||||----------------||--------------||||-----...------
----------------------|||-----%%%%%-|||||----------------||--------------||||-----...------
-------------------%%%%%%%%----|||--|||||----------------||-------%%%----||||-----...------
--------------------||||||-----|||--|||||-------------U-||--------|-----||||-----...------
--------------------||||||--o--|||--|||||----------------||--------|-----||||-----...---F--
-M-----------%%%%---||||||-%%%-|||--|||||----------------||--------|-----||||-----...---#--
XXXXXXXXXXX---||----||||||--|--|||--|||||----%%%%-----%%%%%-%%%%%-|-----||||-----...XXXXXX
XXXXXXXXXXX---||----||||||--|--|||--|||||-----||-------|||---|||--|-----||||-----...XXXXXX
```

Figure 3.1: Original Super Mario Bros Level in ASCII tilemap format. `M` marks Mario's spawn; `X` is solid ground; `S`, `@`, `!` are brick and question blocks; `g` is a Goomba; `F` denotes level exit; `-` is empty air.

3.3.3 Level Validation

Every submitted level undergoes format validation before insertion into the database. The validation procedure, implemented in the backend's `seed.py` module, checks:

1. **Height bounds:** the number of rows must satisfy $10 \leq h \leq 20$.
2. **Width bounds:** the number of columns must satisfy $1 \leq w \leq 250$. In practice, all submitted generators produce levels with widths between 150 and 208 columns; the lower bound of 1 is deliberately permissive and may be tightened in a future revision.
3. **Rectangular shape:** all rows must have identical length.
4. **Character whitelist:** every character must belong to the set of 37 allowed tiles (Table 3.4).

Levels that fail any check are rejected with a descriptive error message. Note that the platform performs *format-level* validation only; there is no automated playability or reachability analysis (e.g. checking that the exit is reachable from the spawn). Content hashes are computed for deduplication.

3.4 Battle and Voting System

The core user experience is the *battle*: a blind A/B test in which the player compares two levels produced by different generators, not knowing which generator produced which level.

3.4.1 Battle Flow

A battle proceeds through five stages:

1. **Battle request.** The frontend calls `POST /v1/battles:next`.

2. **Matchmaking.** The AGIS algorithm (Section 3.5) selects two generators; one random level is drawn from each.

3. **Gameplay.** The player plays both levels sequentially (labelled "Left" and "Right"). During play, the engine records a trajectory of Mario's position and death events.

4. **Voting.** After completing both levels, the player chooses one of four options: *Left is Better*, *Right is Better*, *Tie*, or *Skip*. The "Skip" option, used when the player feels unable to judge or encounters a technical issue, does not affect ratings.

5. **Rating update.** The backend updates Glicko-2 ratings for both generators atomically within a database transaction (Section 3.6).

Figure 3.2 shows the main gameplay interface. After completing both levels, the voting panel (Figure 3.3) allows the player to cast their preference and optionally tag each level.

Figure 3.2: Main gameplay view: the player experiences a procedurally generated level in the browser-based Mario engine.

Figure 3.3: Voting panel shown after both levels are played, with preference buttons and optional tag selection.

3.4.2 Tagging System

In addition to the preference vote, players may optionally tag each level in the battle with up to three descriptive labels drawn from a fixed vocabulary of seven tags:

Tag	Description
`fun`	The level was enjoyable to play
`boring`	The level lacked engagement
`creative`	The level showed novel or interesting design
`too_hard`	The level was excessively difficult
`too_easy`	The level was trivially easy
`impossible`	The level appeared to be uncompletable
`broken_graphics`	The level had visual or rendering issues

Table 3.5: The seven available tags for level characterisation.

Tags are stored as boolean columns in the `votes` table and aggregated in `level_stats`, enabling analyses such as correlating perceived difficulty with win rates or identifying generators whose levels are consistently tagged as `impossible`.

3.5 Matchmaking: AGIS Algorithm

3.5.1 Problem Definition

Given n generators with current Glicko-2 ratings and uncertainties, the matchmaking problem is to select a pair for the next battle that maximises information gain toward accurate final rankings. Naïve strategies are suboptimal:

- **Uniform random** wastes battles on already-converged pairs.
- **Most-uncertain-first** neglects head-to-head comparisons between similarly rated generators.
- **Round-robin** ignores rating information and pairs dissimilar generators.

To address these shortcomings, we designed **AGIS (Adaptive Glicko-Informed Selection)**, a custom two-stage matchmaking algorithm that incorporates rating uncertainty, skill similarity, and pairwise coverage into generator selection. While the individual components draw on well-known ideas from information-theoretic experimental design (uncertainty sampling, coverage balancing), their combination into a lightweight, Glicko-2-aware matchmaker tailored to the PCG evaluation setting is, to our knowledge, novel.

3.5.2 Stage 1: First Generator Selection

Each generator i receives a selection weight w_i composing an uncertainty term and a games-played term:

$$w_i = \left(1 + \widetilde{\mathrm{RD}}_i\right)^2 \cdot \mathrm{boost}(\mathrm{games}_i) \tag{3.1}$$

where $\widetilde{\mathrm{RD}}_i = (\mathrm{RD}_i - \mathrm{RD}_{\min})/(\mathrm{RD}_{\max} - \mathrm{RD}_{\min})$ is the normalised rating deviation. The boost function provides a 4× weight multiplier for generators with zero games, linearly decaying to 1× at `MIN_GAMES` (default: 30). After convergence, a mild quality bias is applied:

$$\mathrm{boost}_{\mathrm{converged}} = 0.8 + q_{\mathrm{bias}} \cdot \min\Big(1,\ \max\big(0,\ \tfrac{\mu_i - 600}{800}\big)\Big) \tag{3.2}$$

where $q_{\mathrm{bias}} = 0.2$ is the configurable quality bias strength. The weights are normalised to sum to 1, forming a categorical (discrete) probability distribution; the first generator is then drawn from this distribution via weighted random sampling.

3.5.3 Stage 2: Second Generator Selection

The intuition behind Stage 2 is that the most informative battle is one between generators of *similar* strength (maximising discriminative power) whose ratings are still *uncertain* (maximising information gain per battle), while ensuring that every pair accumulates enough head-to-head data for a reliable confusion matrix. The four weighted components below formalise these intuitions.

Given the first generator g_1, the second is selected using a composite pair weight:

$$w_{1,j} = \alpha \cdot S(g_1, g_j) \; + \; \beta \cdot U(g_j) \; + \; \gamma \cdot I(g_1, g_j) \; + \; \mathrm{cov}(g_1, g_j) \tag{3.3}$$

The four components are:

- **Rating similarity** $S(g_1, g_j) = \exp\!\big(-\Delta\mu^2 / 2\sigma_s^2\big)$, a Gaussian kernel with $\sigma_s = 150$ penalising large rating gaps.
- **Uncertainty** $U(g_j) = 1 + \widetilde{\mathrm{RD}}_j$, favouring uncertain generators (range $[1, 2]$).
- **Information gain** $I(g_1, g_j)$, incorporating the joint RD and a match-quality estimate.
- **Coverage bonus** $\mathrm{cov}(g_1, g_j)$: for pairs with fewer than `TARGET_BATTLES_PER_PAIR` (default: 10) battles, an exponential bonus $2 \cdot e^{-c/3}$ is applied, where c is the current battle count; otherwise a residual 0.1 is used.

3.5.4 Parameters

Table 3.6 lists the configurable AGIS parameters and their defaults, which are set via environment variables.

Parameter	Description	Default
α	Rating similarity weight	0.5
β	Uncertainty weight	0.3
γ	Information gain weight	0.2
`MIN_GAMES`	Games before "converged"	30
`TARGET_BATTLES_PER_PAIR`	Min. battles per pair	10
`RATING_SIGMA`	Similarity kernel σ_s	150.0
`QUALITY_BIAS`	Quality bias strength q_{bias}	0.2

Table 3.6: AGIS algorithm parameters and default values.

3.6 Glicko-2 Rating System

PCG Arena uses the Glicko-2 rating system [1] to maintain uncertainty-aware rankings. Each generator carries three parameters:

- **Rating (μ):** the skill estimate, displayed on a scale centred at 1000.
- **Rating Deviation (RD, ϕ):** the uncertainty in the rating. High RD indicates an unreliable estimate; low RD indicates a well-established rating.
- **Volatility (σ):** the expected fluctuation in performance over time.

3.6.1 Expected Score

The expected score when generator A faces generator B is:

$$E(\mu_A, \mu_B, \phi_B) = \frac{1}{1 + \exp\big(-g(\phi_B)(\mu_A - \mu_B)\big)} \tag{3.4}$$

where $g(\phi) = 1/\sqrt{1 + 3\phi^2/\pi^2}$ attenuates the prediction when the opponent's rating is uncertain.

3.6.2 Rating Update Procedure

After each battle, the winning generator receives a score of 1, the loser 0, and ties yield 0.5 for both. The update proceeds as follows:

1. Convert ratings to the internal Glicko-2 scale (centred at 0, divided by $\lambda = 173.7178$). This constant equals $400/\ln 10 \approx 173.7178$ and bridges the Elo convention (400 points per tenfold win-probability change) with the natural logistic function used internally by Glicko-2.
2. Compute the variance v from the opponent's RD.
3. Compute $\Delta = v \cdot g(\phi_B)(s - E)$, where s is the actual score.
4. Update volatility σ' via the iterative Illinois algorithm (system constant $\tau = 0.5$).
5. Update RD: $\phi' = 1/\sqrt{1/\phi^{*2} + 1/v}$, where $\phi^* = \sqrt{\phi^2 + \sigma'^2}$.
6. Update rating: $\mu' = \mu + \phi'^2 \cdot g(\phi_B)(s - E)$.
7. Convert back to the display scale and clamp to $[100, 3000]$.

All rating updates are performed atomically within a single database transaction, with an audit record written to the `rating_events` table.

3.6.3 Configuration

Table 3.7 lists the Glicko-2 constants used by PCG Arena.

Parameter	Value
Initial rating (μ_0)	1000
Initial RD (ϕ_0)	350
Initial volatility (σ_0)	0.06
System constant (τ)	0.5
Scale factor (λ)	173.7178
Minimum RD	30
Maximum RD	350
Minimum rating	100
Maximum rating	3000

Table 3.7: Glicko-2 configuration parameters.

The RD of each generator provides a natural 95% confidence interval for its true rating: $\mathrm{CI}_{95\%} = [\mu - 1.96\,\mathrm{RD},\ \mu + 1.96\,\mathrm{RD}]$. However, the public leaderboard displays only the point-estimate rating; confidence intervals are not shown on the platform's frontend. Figure 3.4 shows the leaderboard as seen by players.

Figure 3.4: Public leaderboard ranking generators by their Glicko-2 rating.

Clicking a generator reveals detailed statistics including rating history, per-opponent win rates, a level gallery, and tag distributions (Figure 3.5).

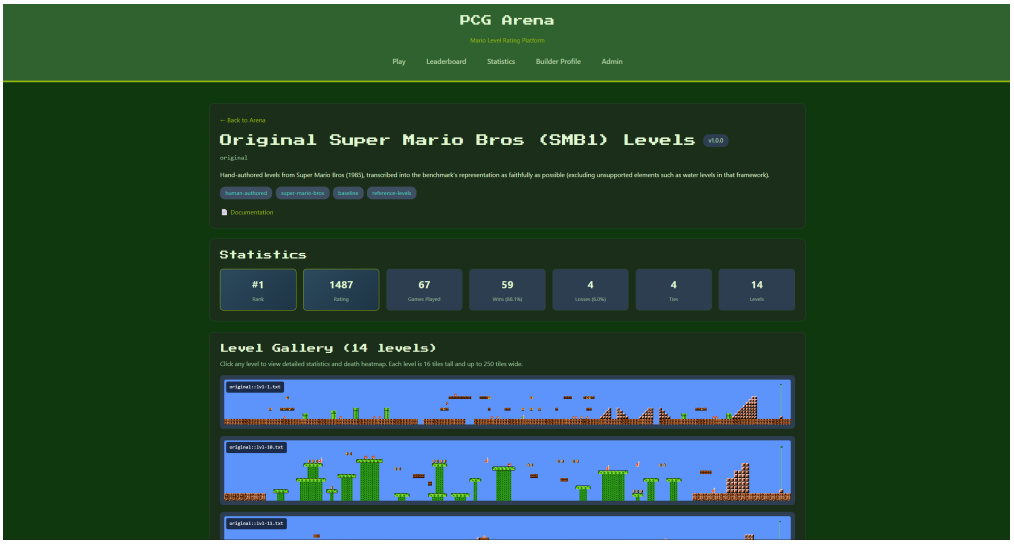

Figure 3.5: Generator detail page showing level gallery and performance statistics.

3.7 Builder Profile and Generator Submission

Authenticated users may submit their own generators through the Builder Profile page. The submission workflow is:

1. The user provides generator metadata: name, version, description, and a URL to the generator's source repository.
2. The user uploads a ZIP archive containing level files in the ASCII tilemap format.
3. The backend validates every level (Section 3.3.3). Levels that fail validation are rejected with detailed error messages.
4. Valid levels and the generator record are inserted into the database.
5. An initial Glicko-2 rating of 1000 ± 350 is assigned.
6. The generator appears on the leaderboard immediately and enters the matchmaking pool.

Each user may own at most three generators. When a user wishes to remove a generator, the system distinguishes two cases. If the generator has already participated in battles, it is *soft-deleted*: marked as inactive and excluded from future matchmaking, but its records (levels, ratings, votes) are retained so that historical analyses and the confusion matrix remain consistent. If the generator has never appeared in a battle, it is *hard-deleted* (fully removed from the database) since no other data depends on it.

Figure 3.6 shows the Builder Profile page.

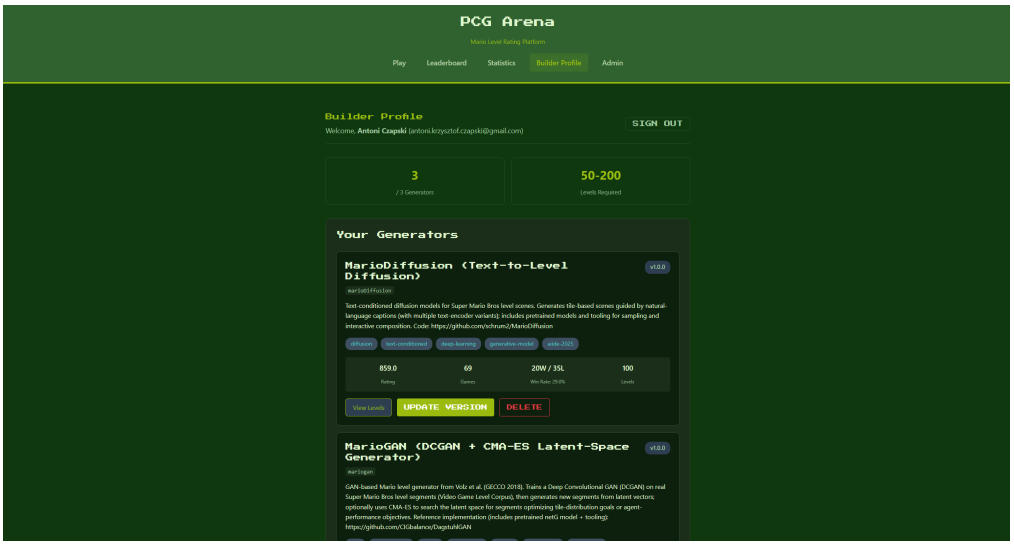

Figure 3.6: Builder Profile page for submitting and managing generators.

3.8 Authentication

PCG Arena supports two authentication methods:

- **Google OAuth 2.0.** The frontend opens a Google sign-in popup; the returned ID token (JWT) is sent to `POST /v1/auth/google`, where the backend verifies it against Google's servers. Google users are automatically marked as email-verified.
- **Email and password.** Users register with an email, password, and display name. Passwords are hashed with bcrypt (work factor 12). A verification email is sent via Twilio SendGrid containing a unique token (64 hex characters, 24-hour expiry).

Upon successful authentication, the backend issues a session cookie (`arena_session`) containing a 256-bit cryptographically random token. Sessions expire after seven days. HTTP-only cookies prevent client-side script access.

Administrative access is granted to users whose email addresses appear in the `ARENA_ADMIN_EMAILS` configuration list.

3.9 Deployment and Infrastructure

PCG Arena is deployed on Google Cloud Platform using a single Compute Engine instance (`e2-micro`, Ubuntu 22.04 LTS). The backend runs inside a Docker container

orchestrated by `docker-compose`, with the SQLite database file bind-mounted from the host for persistence across container rebuilds.

Caddy serves as the reverse proxy, handling:

- Automatic TLS certificate provisioning and renewal (Let's Encrypt).
- HTTP-to-HTTPS redirection.
- Reverse proxying API requests to `localhost:8080`.
- Static file serving for the frontend production build.

The platform is accessible at `https://pcg-arena.com`. The backend binds to `127.0.0.1:8080` and is not exposed to the public internet directly; all external traffic passes through Caddy. Firewall rules allow only ports 80 and 443. Database backups are performed via a scheduled script that copies the SQLite file to a timestamped archive.

Chapter 4

User Study and Exploratory Data Analysis

4.1 Study Design and Data Collection

tbd

4.2 Dataset Overview

tbd

4.3 Generator Rankings and Performance

tbd

4.4 RQ1: What Makes a Good Level?

4.4.1 Difficulty and the Flow Channel Hypothesis

tbd

4.4.2 Structural Variety and Creativity

tbd

4.4.3 Path Freedom and Player Agency

tbd

4.4.4 Hazard Difficulty and Leniency

tbd

4.5 RQ2: Player Preference Consistency

4.5.1 Preference Clusters

tbd

4.5.2 Skill–Consistency Relationship

tbd

4.6 RQ3: Tag Semantics and Validation

4.6.1 Tag–Telemetry Correspondence

tbd

4.6.2 Feature Importance for Tags

tbd

4.6.3 Predictive Modelling of Preferences

tbd

4.7 Trajectory Analysis

tbd

4.8 Summary of Findings

tbd

Chapter 5

MarioDPO — Preference-Aligned Level Generation

5.1 Motivation and Approach

tbd

5.2 Judge Function Development

tbd

5.2.1 Verticality Reward (Experiment A)

tbd

5.2.2 Hazard Hierarchy (Experiment B)

tbd

5.2.3 Style Matching (Experiment D)

tbd

5.2.4 Final Judge Function

tbd

5.3 Training Pipeline

5.3.1 Supervised Fine-Tuning (SFT)

tbd

5.3.2 Preference Dataset Construction

tbd

5.3.3 DPO Training

tbd

5.4 Column-Major Tokenisation

tbd

5.5 Results

tbd

5.6 Analysis and Discussion

tbd

Chapter 6

Conclusions and Future Work

6.1 Summary of Contributions

tbd

6.2 Limitations

tbd

6.3 Future Directions

tbd

Bibliography

[1] Mark E. Glickman. Example of the Glicko-2 system. Technical report, Boston University, 2012. URL `http://www.glicko.net/glicko/glicko2.pdf`.

[2] Sergey Karakovskiy and Julian Togelius. The Mario AI benchmark and competitions. In *IEEE Transactions on Computational Intelligence and AI in Games*, volume 4, pages 55–67, 2012.